RELATÓRIO – Iteração 5

Implementação - Incremento 1

Projeto: MultiPower: organização e otimização do carregamento de veículos eléctricos.

Grupo: 119809: Henrique Moreira Moreira
120440: João Francisco Ribeiro de Matoso Cardoso
119744: Nicolas Botelho de Sousa

Data/versão: 2025/12/12, v1

Sumário:
A MultiPower é uma plataforma de mobilidade elétrica desenvolvida para centralizar e simplificar a gestão e utilização de estações de carregamento.

A aplicação oferece uma experiência integrada ao condutor, combinando sistemas de geolocalização (via API OpenChargeMap) para mapeamento de estações em tempo real com funcionalidades robustas de gestão pessoal. O sistema permite a administração completa de contas de utilizador, gestão de frota pessoal através de uma "Garagem Virtual" e controlo financeiro detalhado mediante uma Carteira Digital, que inclui visualização gráfica de consumos e histórico de transações.

Tecnologicamente, a solução adota uma arquitetura Cliente-Servidor, utilizando uma API REST em Node.js, persistência de dados local em SQLite3 e uma interface web dinâmica e responsiva.

Principais resultados :

- ➔ Um mapa interativo para visualização de estações de carregamento (OpenChargeMap e locais) com disponibilidade e estado.
- ➔ Gestão Financeira (Wallet): Implementação de uma carteira digital funcional com consulta de saldo em tempo real e visualização gráfica de consumos através da biblioteca Chart.js.
- ➔ Garagem Virtual: Desenvolvimento de um sistema dinâmico para adicionar e remover veículos associados à conta do utilizador, com persistência de dados.
- ➔ Operações de Carregamento: Funcionalidades completas para reservar estações (com débito de saldo virtual) e controlar o início e paragem dos carregamentos diretamente na app.

Conteúdo:

1. Introdução....2
1.1. Sumário executivo....2
1.2. Controlo de versões....2
2. Arquitetura do sistema....3
2.1. Objetivos gerais....3
2.2. Requisitos com impacto na arquitetura....3
2.3. Decisões e justificação....4
2.4. Arquitetura do software....6
2.5. Integração de sistemas....7
3. Incremento 1....8
3.1. Histórias de utilização selecionadas....8
3.2. Estratégia e estado da implementação....14
3.3. Aceitação e garantia de qualidade....16
4. Bibliografias....16

1. Introdução

1.1. Sumário executivo

Este relatório apresenta os resultados da construção, adaptando os resultados esperados nas etapas de Elaboration e Construction do método OpenUP.

O primeiro incremento foca a validação da arquitetura proposta e das histórias nucleares. Foram consideradas sobretudo as funcionalidades relacionadas com a Descoberta de Estações, Gestão Financeira e Administração de Veículos.

Como apresentado anteriormente, visamos criar um núcleo robusto do sistema, com o intuito de garantir a entrega de uma primeira implementação funcional. Para tal, o foco recaiu sobre a descoberta e preparação para recarga (UC1, UC4), materializando um MVP (Minimum Viable Product) que assegura as funcionalidades essenciais de Busca e Seleção, Gestão Financeira e Suporte Operacional.

1.2. Controlo de versões

Quando?	Responsável	Alterações significativas
15/12/2025	Henrique moreira	Realização do ponto 3
18/12/2025 - 19/12/2025	Nicolas Sousa	Realização do ponto 2
19/12/2025	João Cardoso	Revisão estrutural e ortográfica do relatório.

Tabela 1 - Controlo de versões

2. Arquitetura do sistema

2.1. Objetivos gerais

O sistema MultiPower foi pensado para resolver desafios atuais do ecossistema de postos de carregamento de carros elétricos. Foram, desse modo, definidos os requisitos funcionais e não funcionais do produto. O objetivo central é suportar uma plataforma capaz de integrar serviços e intervenientes essenciais em um único ecossistema digital unificado.

Entre os objetivos principais está a integração de sistemas externos e legados, de forma que a plataforma se comunique com infraestruturas já existentes de diferentes prestadores de postos elétricos. Além disso, é importante a integração com outros sistemas externos, como serviços de geolocalização e pagamentos. Com essas medidas, a interoperabilidade é garantida, e a evolução futura do sistema torna-se mais simples.

O desempenho é outro aspecto crítico, especialmente em operações sensíveis ao tempo, como pesquisa de carregadores próximos, consulta de disponibilidade em tempo real,

reservas e pagamentos. O sistema deve oferecer respostas rápidas mesmo com alta concentração de usuários, sem comprometer seu funcionamento.

A arquitetura é modular, organizada para garantir separação de responsabilidades, escalabilidade e facilidade de manutenção futura. O frontend é responsável pela interface do usuário; o backend processa solicitações, integra serviços externos e gerencia a base de dados, que armazena informações de forma segura. APIs externas fornecem dados atualizados em tempo real, garantindo interoperabilidade entre sistemas.

Essa abordagem modular permite que os componentes sejam desenvolvidos e testados de forma independente, assegurando que os requisitos de desempenho, disponibilidade e interoperabilidade sejam atendidos de maneira eficiente.

2.2. Requisitos com impacto na arquitetura

Requisito	Descrição
RNF.4	A aplicação deve poder funcionar corretamente em dispositivos de gama média, garantindo fluidez e estabilidade em equipamentos com, pelo menos, 3 GB de RAM e Android 10 ou iOS 13 em diante.
RInt.3	A aplicação tem de permitir a configuração de alertas e notificações personalizadas (incluindo bateria fraca, promoções ativas e aviso de conclusão do carregamento).
RDes.1	A pesquisa de estações de carga não deve demorar mais de 3 segundos em condições normais de ligação à internet.
RDes.2	O processo de pagamento deve ser concluído em menos de 10 segundos após a confirmação do utilizador.
RDes.3	O mapa de estações deve atualizar-se automaticamente a cada 30 segundos para refletir a disponibilidade em tempo real.
RIntExt.1	O sistema deve integrar-se com APIs públicas de estações de carga (Mobi.e, Galp, Shell Recharge, etc.) para obter disponibilidade atualizada.
RIntExt.2	Os pagamentos devem realizar-se através de serviços externos certificados como MBWay, PayPal ou cartões de crédito.
RI.1	A aplicação deve ser desenvolvida para Android e iOS como plataformas de lançamento principais.

Tabela 2 - “Repescagem” de requisitos com impactos importantes no primeiro incremento.

2.3. Decisões e justificação

Tendo em conta os objetivos definidos para a arquitetura do sistema MultiPower, bem como os requisitos funcionais e não funcionais identificados na fase de Análise, foram tomadas as seguintes decisões arquiteturais e tecnológicas:

Frontend desenvolvido com HTML5, CSS3 e JavaScript (Vanilla).
Estas tecnologias foram escolhidas por serem amplamente suportadas pelos navegadores modernos e permitirem o desenvolvimento de interfaces leves, responsivas e eficientes. A utilização de JavaScript puro reduz dependências externas, melhora o controlo sobre a lógica da aplicação e garante um bom desempenho em dispositivos de gama média.

Visualização de dados realizada com a biblioteca Chart.js.
A Chart.js foi selecionada por permitir a criação de gráficos interativos de forma simples e com baixo overhead. Esta biblioteca responde ao requisito de fornecer ao utilizador uma análise clara do consumo energético e da variação de preços, contribuindo para a tomada de decisões informadas.

Geolocalização e mapas implementados com Leaflet.js e a API OpenChargeMap.
O Leaflet.js foi escolhido por ser uma biblioteca leve, de fácil integração e com boa performance na renderização de mapas interativos. A utilização da API pública OpenChargeMap permite obter dados atualizados sobre estações de carregamento, garantindo interoperabilidade e reduzindo a dependência de soluções proprietárias.

Backend desenvolvido em Node.js com o framework Express.
O Node.js foi adotado devido à sua capacidade de lidar eficientemente com operações assíncronas e elevado número de pedidos simultâneos. O framework Express simplifica a criação da API REST, facilitando a gestão de rotas e a organização da lógica de negócio do sistema.

Persistência de dados assegurada através de SQLite3.
O SQLite3 foi selecionado por ser uma solução leve, simples de implementar e adequada ao volume de dados previsto para o projeto. Esta base de dados permite o armazenamento seguro das informações dos utilizadores, veículos e histórico de transações, mantendo conformidade com o modelo de domínio definido.

Conclusão das Decisões Arquiteturais
As decisões tecnológicas adotadas foram diretamente orientadas pelos requisitos com impacto na arquitetura, nomeadamente os requisitos de desempenho, interoperabilidade e compatibilidade. A escolha de tecnologias leves e bem dominadas permite garantir tempos de resposta reduzidos, estabilidade em dispositivos de gama média e integração eficiente com APIs externas, o que responde aos requisitos **RNF** e **RDes** identificados.

Ao optar por soluções cuja implementação é plenamente compreendida pela equipa, foi possível assegurar um maior controlo sobre o comportamento do sistema, reduzindo complexidade desnecessária e riscos associados a tecnologias mais avançadas, mas pouco dominadas.

2.4. Arquitetura do software

O sistema MultiPower possui uma arquitetura modular, onde cada módulo desempenha funções específicas e interage com os demais para garantir o funcionamento integrado da plataforma.

Frontend:

- ➔ Responsável pela interface do usuário, apresenta informações de forma intuitiva e permite interações como pesquisa de carregadores, reservas e pagamentos.
- ➔ Comunica-se diretamente com o backend para enviar solicitações e receber dados atualizados em tempo real.

Backend:

- ➔ Processa todas as solicitações provenientes do frontend e gerencia a lógica de negócios do sistema.
- ➔ Integra serviços externos, como sistemas de geolocalização e plataformas de pagamento, e se conecta à base de dados para armazenar ou recuperar informações.
- ➔ Coordena a comunicação entre diferentes módulos, garantindo que cada operação seja executada corretamente.

Base de dados:

- ➔ Armazena informações de forma segura, incluindo dados de usuários, status dos carregadores, reservas e histórico de transações.
- ➔ Fornece dados estruturados para o backend e garante que as consultas sejam rápidas, mesmo em alta demanda.

APIs externas:

- ➔ Fornecem dados em tempo real sobre disponibilidade de carregadores, localização e outros serviços essenciais.
- ➔ Permitem a interoperabilidade entre sistemas legados e externos, tornando o MultiPower compatível com múltiplos prestadores de serviços.

Colaboração entre os módulos:

- ➔ O frontend solicita informações ao backend, que processa a lógica de negócios e consulta a base de dados ou APIs externas conforme necessário.
- ➔ As APIs externas garantem que o sistema tenha dados atualizados, enquanto o backend centraliza e organiza essas informações para o frontend.
- ➔ A modularidade permite que cada componente funcione de forma independente, mas de maneira coordenada, garantindo desempenho, escalabilidade e fácil manutenção futura.

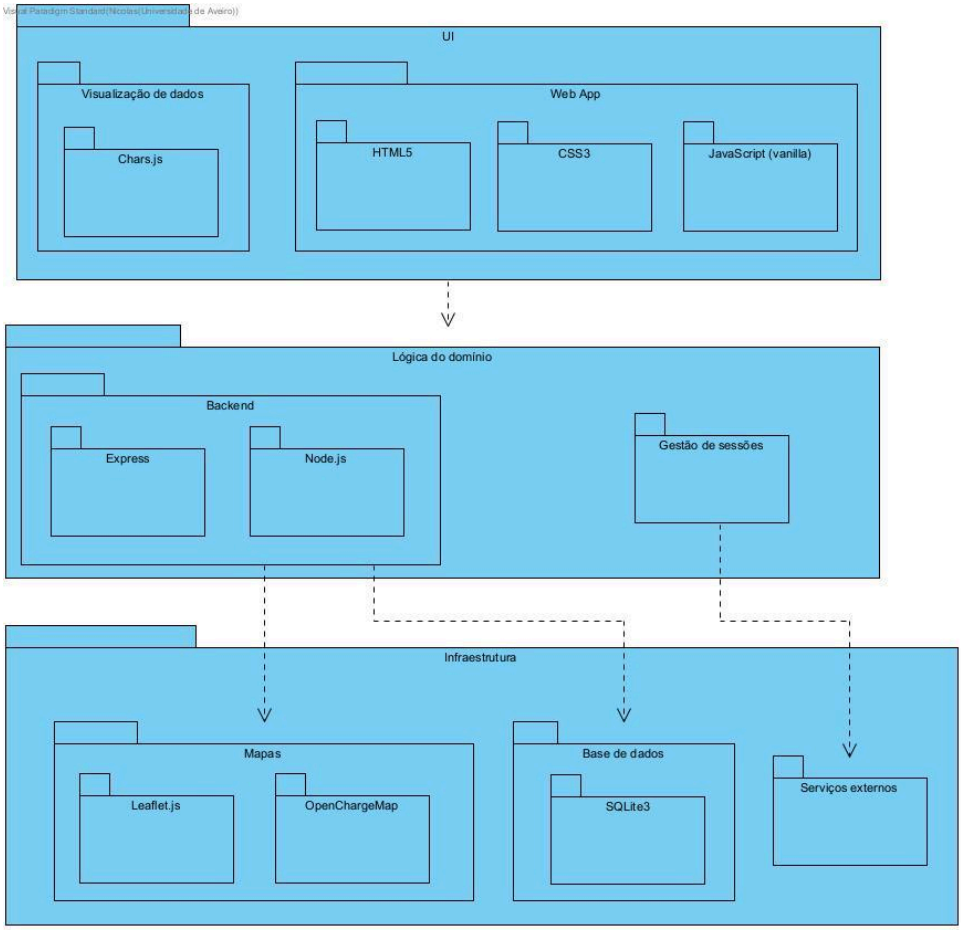


Figura 1 - Diagrama de pacotes em vista lógica (módulos).

2.5. Integração de sistemas

Sistema de pagamento

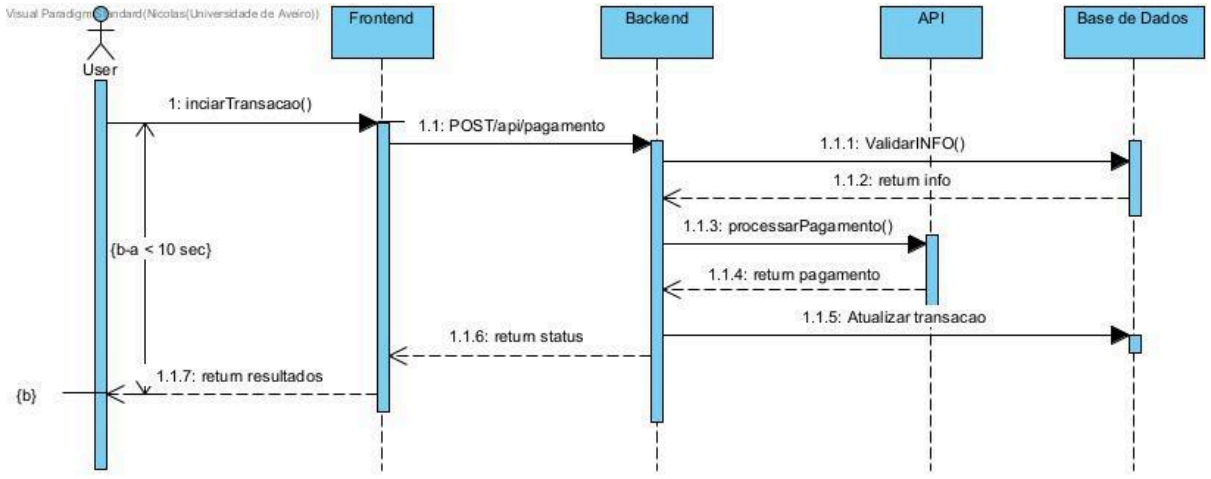


Figura 2 - Diagrama de atividades do sistema de pagamento da MultiPower

Sistema de carregamento

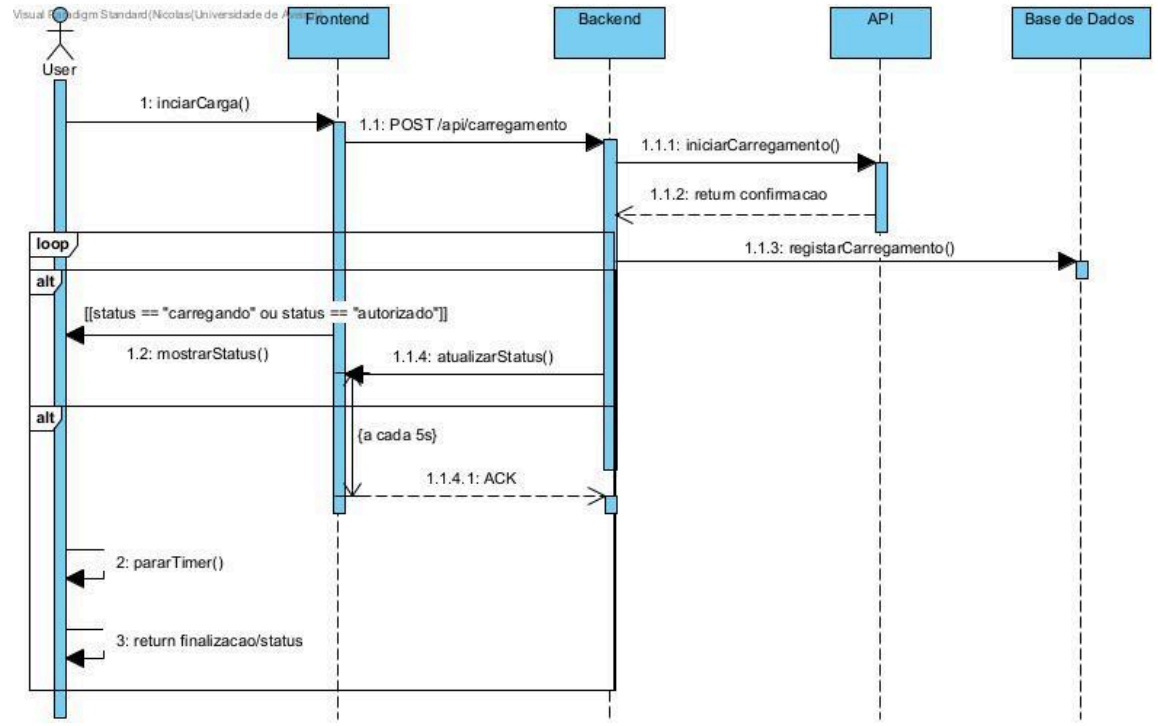


Figura 3 - Diagrama de atividades do sistema de carregamento de um veículo elétrico e sua interação com os módulos.

Sistema de busca de postos

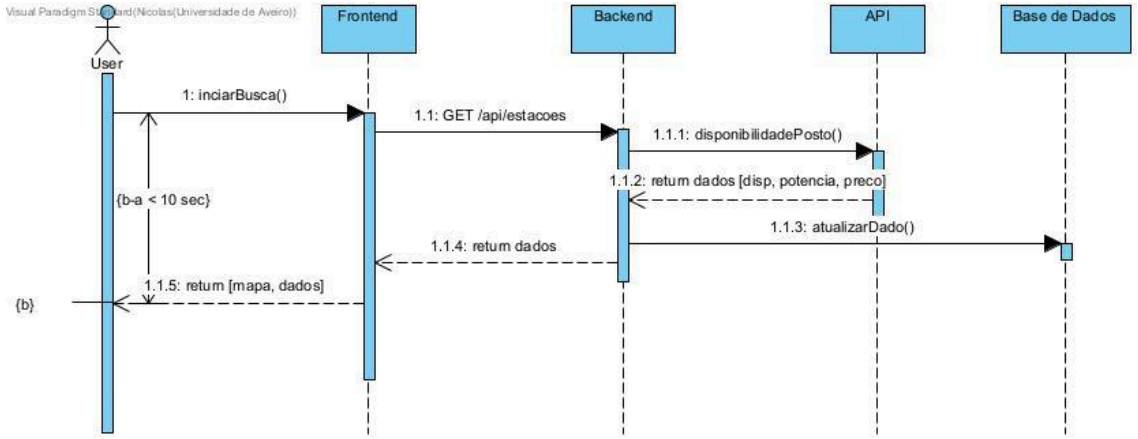


Figura 4 - Diagrama de atividades do sistema de busca de postos.

3. Incremento 1

3.1. Histórias de utilização selecionadas

Como dito anteriormente, visamos um primeiro incremento, desta forma, buscamos garantir as principais funcionalidades e características principais do aplicativo.
Foi escolhido os seguintes casos de utilização:

- UC 1.1: Pesquisar posto
- UC 1.2: Guiar ao posto
- UC 1.3: Booking de carregadores
- UC 1.4: Carregamento do carro
- UC 4.1: Efetuar pagamento do carregamento
- UC 4.2: Ver histórico de pagamentos
- UC 4.3: Gerir método de pagamento

Acreditamos que esses são os principais casos de utilização necessários para a aplicação, pois mostra os pontos centrais da aplicação, seu funcionamento e direcionamento da aplicação num nível bem básico e nuclear.

História / Use Case	**Critérios de aceitação**
UC 1.1: O Condutor pesquisa postos de carregamento Sendo eu um **Condutor**, Quero **visualizar e pesquisar estações** no mapa interativo, De modo a encontrar um local disponível para carregar o meu veículo.	**Cenário 1: Visualização com sucesso** **Dado** que estou na página "Mapa" da aplicação **E** a minha localização está ativa **Quando** o mapa carrega **Então** vejo marcadores das estações da rede OpenChargeMap e locais **E** consigo distinguir quais estão "Disponíveis" ou em "Manutenção".

	Cenário 2: Consulta de detalhes **Dado** que vejo um marcador no mapa **Quando** clico sobre o ícone da estação **Então** vejo um popup com a morada, tipo de conector e estado atual.
UC 1.2: O Condutor guia-se até ao posto Sendo eu um **Condutor**, Quero **obter a localização exata** do posto, De modo a deslocar-me fisicamente até lá.	**Cenário 1: Identificação de rota** **Dado** que selecionei um posto no mapa **Quando** verifico os detalhes do posto **Então** a morada e as coordenadas são apresentadas corretamente para navegação.
UC 1.3: Reserva de posto (Booking) Sendo eu um **Condutor**, Quero **reservar uma estação** antecipadamente De modo a garantir que tenho lugar quando chegar.	**Cenário 1: Reserva com saldo suficiente** **Dado** que selecionei um posto disponível **E** tenho saldo positivo na minha Wallet **Quando** clico no botão "Reservar" **Então** o sistema debita o custo estimado/reserva

	E o estado do posto muda para "Ocupado/Reservado".
UC 1.4: Realizar carregamento Sendo eu um **Condutor**, Quero **iniciar e parar o carregamento** pela app, De modo a controlar o abastecimento do veículo.	**Cenário 1: Iniciar carregamento** **Dado** que estou no posto reservado **Quando** clico em "Iniciar Carregamento" **Então** o sistema começa a contabilizar o tempo/energia. **Cenário 2: Terminar carregamento** **Dado** que tenho um carregamento a decorrer **Quando** clico em "Parar Carregamento" **Então** a sessão termina e o custo final é calculado.
UC 4.1: Efetuar pagamento (via Wallet) Sendo eu um **Condutor**, Quero **pagar o serviço** automaticamente, De modo a não perder tempo com terminais físicos.	**Cenário 1: Pagamento automático** **Dado** que terminei uma sessão de carregamento ou reserva **Quando** o valor final é apurado **Então** o montante é subtraído automaticamente do meu saldo virtual (Wallet) **E** recebo uma mensagem de sucesso.

UC 4.2: Ver histórico de pagamentos Sendo eu um **Condutor**, Quero **consultar os meus movimentos**, De modo a saber quanto gastei em carregamentos.	**Cenário 1: Consulta de extrato** **Dado** que estou na área "Conta" ou "Wallet" **Quando** clico em "Transações" **Então** vejo uma lista ordenada com a data, valor e descrição dos carregamentos passados.
UC 4.3: Gerir saldo (Carteira Digital) Sendo eu um **Condutor**, Quero **adicionar saldo** à minha conta, De modo a ter fundos para futuros carregamentos.	**Cenário 1: Carregamento de saldo** **Dado** que estou na página "Wallet" **Quando** clico no botão de adicionar saldo e insiro um valor **Então** o meu "Saldo Atual" é atualizado imediatamente com o novo montante.

Tabela 3 - Histórias de utilização com importância no backlog desta interação.

3.2. Estratégia e estado da implementação

A plataforma MultiPower foi materializada através de uma arquitetura modular que converteu os requisitos conceptuais em funcionalidades operacionais.

O que foi implementado

Mapa Interativo e Geolocalização: Implementação do sistema de pesquisa de postos que integra a API OpenChargeMap com estações locais. O utilizador pode visualizar a localização, disponibilidade em tempo real e o estado de manutenção de cada ponto de carga diretamente no mapa.

Gestão de Carregamentos e Reservas: Através do sistema de reservas, o cliente pode garantir a utilização de um posto num horário específico. A aplicação permite iniciar e parar sessões de carregamento, com o respetivo custo a ser processado automaticamente.

Autenticação e Perfil do Utilizador: Sistema de gestão de conta que permite o registo, login e a manutenção de dados pessoais e credenciais de segurança (password).

Gestão Financeira e Saldo: Desenvolvimento da Carteira Digital (Wallet), onde o usuário consulta e adiciona saldo virtual. O sistema gere automaticamente o débito de valores para reservas e carregamentos, assegurando a integridade dos dados financeiros.

Histórico de Transações: Área dedicada à consulta detalhada do histórico de carregamentos e pagamentos realizados, garantindo a transparência dos consumos efetuados.

Garagem Virtual (Gestão de Veículos): Implementação do módulo de gestão de frota pessoal , permitindo ao condutor adicionar, editar ou remover veículos elétricos da sua conta.

Painel de Administração: Funcionalidades reservadas ao administrador da app para a gestão da infraestrutura, permitindo a adição/remoção de estações locais e a atualização do estado de manutenção dos postos

As principais ferramentas de Implementação

Frontend (Interface e Lógica de Cliente): Foi utilizado HTML5 para a estruturação, CSS3 para o design responsivo e JavaScript (Vanilla) para a lógica dinâmica.

Visualização de Dados: A biblioteca Chart.js foi integrada na página da carteira para gerar gráficos de consumo mensal e variação de preços, cumprindo o objetivo de fornecer consultoria e análise de eficiência energética.

Geolocalização: O mapa interativo foi desenvolvido com a biblioteca Leaflet.js, permitindo a visualização de estações de carregamento através da integração com a API externa OpenChargeMap.

Backend (Servidor): A lógica do servidor foi construída em Node.js com o framework Express, gerindo os pedidos da API REST (como GET /api/user e POST /api/carros/add).

Base de Dados: Utilizou-se SQLite3 para a persistência de dados local, armazenando de forma segura as informações dos utilizadores, veículos e histórico de transações em conformidade com o modelo de domínio

O que está em falta

Assistente Virtual (IA) (parcial): Ocasionalmente, o chatbot apresenta dificuldades em manter o contexto da conversa ou latência na resposta devido à dependência de serviços externos.

Otimização de Rotas: O algoritmo de navegação sugere o trajeto, mas nem sempre seleciona a rota mais rápida, uma vez que o sistema atual ainda não integra dados de trânsito em tempo real.

3.3. Aceitação e garantia de qualidade

No portefólio do grupo, encontram-se disponíveis testes que demonstram o funcionamento do projeto

4. Bibliografias

https://es.scribd.com/document/449807594/Metodologia-Open-UP

https://stackshare.io